

Implementation of Multi-Query RAG Query Expansion + Parallel Retrieval + Reciprocal Rank Fusion E-Commerce Analytics System — Olist Brazilian Dataset RAGAS & DeepEval Evaluation Framework | May 2026

Abstract

This document describes the end-to-end implementation of a Multi-Query RAG system built on the Olist Brazilian e-commerce dataset. The system addresses the vocabulary mismatch problem in standard dense retrieval by generating N-1 paraphrased query variants using a Groq LLM, retrieving documents for each variant independently from ChromaDB, and fusing the ranked lists using Reciprocal Rank Fusion (RRF). The top-5 fused documents are passed to *llama-3.3-70b-versatile* for answer generation. The system makes 2 Groq calls per query: one for expansion (temperature=0.7) and one for generation (temperature=0.1).

1. System Overview

Multi-Query RAG solves a fundamental limitation of single-query retrieval: the exact phrasing of a user's question may not match the vocabulary used in the knowledge base. By generating multiple semantically equivalent formulations of the same question, the system casts a wider semantic net — different paraphrases activate different embedding dimensions, retrieving complementary sets of relevant documents. Reciprocal Rank Fusion then merges these per-query ranked lists into a single

fused ranking, promoting documents that appear consistently across multiple query formulations.

1.1 Architecture Diagram

Layer	Component	Technology	Role
Data	Raw Olist CSV Files	9 CSV files / 99K-1M rows each	Source data for KB construction
Processing	ETL Pipeline (5 steps)	Python / Pandas	Join, enrich, aggregate to KB docs
Knowledge	kb_all_documents.json	13,225 JSON documents	Structured KB across 6 document types
Indexing	ChromaDB Vector Store	all-MiniLM-L6-v2 (384-dim)	Cosine-similarity semantic index
Expansion	query_expander.py	llama-3.3-70b-versatile (temp=0.7)	Generate 3 paraphrase variants + original
Retrieval	retriever.py (multi)	ChromaDB, top-10 per query variant	4x parallel variant retrieval
Fusion	fusion.py (RRF)	Reciprocal Rank Fusion, k=60	Merge 4 ranked lists, return top-5
Generation	generator.py	llama-3.3-70b-versatile (temp=0.1)	Context-grounded answer synthesis
Evaluation	evaluation/ scripts	RAGAS + DeepEval + golden dataset	Reference-based 0-LLM-judge metrics

2. Data Preparation Pipeline

Multi-Query RAG shares the same Olist knowledge base, ETL pipeline, and ChromaDB vector store as Naive RAG, HyDE RAG, Hybrid RAG, and Reranking RAG. The collection ecommerce_kb is reused — no re-ingestion is required if the chroma_db/ directory already exists.

2.1 ETL Steps

Step	Script	Output	Description
1	step1_load_raw_data.py	9 validated DataFrames	Load CSVs, validate schema, report nulls
2	step2_join_datasets.py	master_joined.csv	Star-schema join on order_id / customer_id / product_id
3	step3_enrich_master.py	master_enriched.csv	Add delivery_days, late_flag, review_score_category
4	step4_build_knowledge_base.py	kb_all_documents.json	Aggregate to 6 document-type layers (13,225 docs)
5	step5_build_golden_dataset.py	golden_dataset.csv	Generate 100 Q&A pairs via Gemini Flash LLM

2.2 Knowledge Base Document Types

Layer	ID Pattern	Count	Fields
Order	order_<uuid>	~99K	Order ID, payment, delivery dates, review, freight
Seller	seller_<id>	~3K	Seller city/state, avg review, total revenue, late rate

Layer	ID Pattern	Count	Fields
Product Category	category_<name>	~73	Category name, total orders, avg review, revenue, late rate
Delivery Status	delivery_status_<type>	3	early / on_time / late — aggregate stats per group
Customer State	state_<abbr>	~27	State-level orders, satisfaction, logistics metrics
Temporal	month_<YYYY_MM>	~25	Month-level: total orders, avg payment, top categories

3. Query Expansion Module

The query expander is the unique component of Multi-Query RAG. Given an original query, it asks the LLM to generate N-1 semantically equivalent paraphrases, then prepends the original as the first element. The result is a list of N distinct query formulations that express the same information need using different vocabulary and sentence structure.

3.1 Expansion Algorithm

```
expand_query(query, n=4, temperature=0.7) -> list[str]:
1. n_generated = max(1, n - 1) # = 3 paraphrases
2. System prompt: 'You are a search query expert. Generate {n_generated}
   semantically equivalent but differently phrased versions of the query.'
3. Call Groq LLM (temperature=0.7) -> numbered list of paraphrases
4. Parse numbered list: strip '1. ', '2. ' prefixes
5. Deduplicate by lowercased key
6. Prepend original query as element [0]
7. Return unique[:n]

Fallback (any exception): return [query] * n

Example output for 'What is the average delivery time?':
[0] 'What is the average delivery time?' (original)
[1] 'How long does it take for orders to arrive?'
[2] 'What is the typical shipping duration?'
[3] 'How many days until delivery on average?'
```

3.2 Expansion Configuration

Parameter	Value	Notes
NUM_QUERY_VARIANTS	4	1 original + 3 LLM-generated paraphrases
EXPANDER_TEMPERATURE	0.7	Higher than generation temp (0.1) to maximise vocabulary diversity
LLM model	llama-3.3-70b-versatile	Same model as generation to reduce cold-start overhead
Max tokens	300	Sufficient for 3 short paraphrase sentences
Fallback	[query]*n	Returns n copies of original if LLM call fails
Original position	Index 0	Original always prepended — never lost even if LLM fails

3.3 Why Higher Temperature for Expansion?

Generation uses temperature=0.1 to produce factual, deterministic answers. Expansion uses temperature=0.7 to maximise vocabulary diversity among paraphrases — if the temperature were low, all paraphrases would be near-identical rewrites of the original, providing little additional retrieval coverage. High diversity in query vocabulary ensures that different paraphrases activate different embedding dimensions, expanding the semantic coverage of the retrieval step.

4. Multi-Retrieval Module

The retriever runs one ChromaDB cosine-similarity query per expanded query variant, collecting top-10 documents per variant. This produces 4 independent ranked lists (one per query variant), each containing up to 10 documents. Documents may appear in multiple lists if they are relevant to multiple query formulations.

4.1 Retrieval API

```
retrieve_for_query(query, top_n=10) -> list[dict]:  
Single ChromaDB query; returns top_n docs with id/text/metadata/distance  
  
retrieve_multi(queries, top_n=10) -> dict[str, list[dict]]:  
{query_variant: retrieve_for_query(query_variant, top_n) for q in queries}  
Returns a dict mapping each query variant string to its doc list.  
  
PER_QUERY_TOP_K = 10 (config.py)  
Total candidates before fusion: up to 4 x 10 = 40 docs (with overlaps)
```

4.2 Why 10 Documents Per Variant?

Fetching 10 documents per query variant (vs 5 for Naive RAG) balances recall against redundancy. With 4 query variants, the union of retrieved documents can contain up to 40 unique documents, giving RRF a rich candidate pool. Fetching fewer (e.g., 5) risks missing relevant documents that only appear in the lower ranks for a specific query formulation. Fetching more (e.g., 20) increases noise in the fusion step without meaningful recall gains for a 13K document corpus.

5. Reciprocal Rank Fusion (RRF)

RRF merges multiple ranked lists into a single fused ranking by accumulating reciprocal rank scores across all lists. A document appearing at rank r in a list contributes $1/(k+r)$ to its total RRF score, where $k=60$ is a smoothing constant. Documents appearing in multiple lists accumulate scores from each appearance — consistently relevant documents rise to the top regardless of which specific query formulation retrieved them.

5.1 RRF Algorithm

```
rff_fuse(ranked_lists, k=60, top_n=5) -> list[dict]:  
  
scores = {} # doc_id -> cumulative RRF score  
best = {} # doc_id -> doc copy with lowest cosine distance  
  
for ranked_list in ranked_lists: # 4 lists  
    for rank, doc in enumerate(ranked_list, start=1):
```

```

doc_id = doc['id']
scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
if doc_id not in best or doc['distance'] < best[doc_id]['distance']:
    best[doc_id] = doc # keep copy with lowest cosine distance

sorted_ids = sorted(scores, key=lambda d: scores[d], reverse=True)
result = []
for doc_id in sorted_ids[:top_n]:
    enriched = dict(best[doc_id])
    enriched['rrf_score'] = round(scores[doc_id], 6)
    result.append(enriched)
return result

```

5.2 RRF Configuration

Parameter	Value	Notes
RRF_K	60	Standard constant from Cormack et al. 2009; reduces sensitivity to top-rank dominance
FINAL_TOP_K	5	Documents passed to LLM after fusion
Deduplication	by id	When doc appears in multiple lists, only one copy kept (lowest cosine distance)
rrf_score	added	Each returned doc gets rrf_score field; higher = more consistently relevant

5.3 RRF Score Example

A document appearing at rank 1 in all 4 query variant lists receives score: $4 \times 1/(60+1) = 4/61 = 0.0656$. A document appearing only at rank 1 in one list receives score: $1/(60+1) = 0.0164$. A document appearing at rank 3 in all 4 lists receives score: $4 \times 1/(60+3) = 4/63 = 0.0635$. RRF thus strongly rewards cross-variant consistency over single-variant top ranking.

Document	List A rank	List B rank	List C rank	List D rank	RRF Score
doc_cat_electronics	1	2	1	3	0.0624 (appears in 4/4 lists)
doc_state_SP	3	-	2	-	0.0316 (appears in 2/4 lists)
doc_month_2017_03	-	1	-	-	0.0164 (appears in 1/4 lists)

6. Answer Generation Module

The generator receives the original query (not the expanded variants) and the top-5 RRF-fused documents. It formats a standard RAG prompt and calls **llama-3.3-70b-versatile** via the Groq API at temperature=0.1 for deterministic, factual responses.

6.1 Prompt Template

System: You are a helpful e-commerce data assistant.
Answer questions using only the provided context.
If the answer cannot be found in the context, say so clearly.

User: Context:
[Document 1]: {fused_docs[0]['text']} # highest rrf_score
[Document 2]: {fused_docs[1]['text']}
... (top-5 RRF-fused documents)

Question: {original_query} # NOT the expanded variants
Answer:

6.2 LLM Configuration

Parameter	Value	Rationale
Model	llama-3.3-70b-versatile	State-of-the-art open model via Groq
Temperature	0.1	Low randomness for factual e-commerce answers
Max tokens	512	Sufficient for analytical answers
Groq calls/q	2	1 for expansion + 1 for generation per query

7. End-to-End Pipeline

7.1 Pipeline Execution Flow

Step	Function	Input -> Output
0 — Init	build_vector_store()	kb_all_documents.json -> ChromaDB collection
1 — Expand	expand_query(query, n=4)	1 query -> [original + 3 paraphrases]
2 — Retrieve	retrieve_multi(expanded, top_n=10)	4 queries -> dict{query: [10 docs]}
3 — Fuse	rrf_fuse(query_results.values(), top_n=5)	4 ranked lists -> 5 fused docs with rrf_score
4 — Generate	generate(original_query, fused_docs)	query + 5 docs -> answer string
5 — Return	run_multiquery_rag(query)	query -> {query, expanded_queries, query_results, retrieved_docs, answer}

7.2 Pipeline Return Schema

```
run_multiquery_rag(query) returns:
{
  'query' : str – original question
  'expanded_queries' : list[str] – [original, paraphrase1, paraphrase2, paraphrase3]
  'query_results' : dict[str, list] – {variant: [10 docs]} for each variant
  'retrieved_docs' : list[dict] – top-5 RRF-fused docs
  each doc has: id, text, metadata,
  distance (cosine),
  rrf_score (fusion score)
  'answer' : str – LLM-generated answer
}
```

7.3 Interactive Entry Point

```
$ python run_multiquery_rag.py
```

Question: What is the average delivery time for SP customers?

Expanded queries:

```
[0] What is the average delivery time for SP customers?
[1] How long does it take to deliver orders to Sao Paulo?
[2] What is the mean shipping duration for state SP?
[3] How many days do customers in SP wait for their orders?
```

Retrieved Documents (after RRF fusion):

```
[1] rrf=0.062 dist=0.38 state_SP
[2] rrf=0.031 dist=0.44 month_2018_08
...
```

Answer: The average delivery time for SP customers is 8.34 days.

8. Groq API Key Management

Multi-Query RAG makes 2 Groq calls per query (expand + generate), doubling the per-query token consumption compared to Naive RAG. The inter-query delay is increased to 10–25 seconds (vs 5–20s for single-call RAGs) to respect per-minute token limits across all 5 parallel evaluation threads.

Key Slot	Query Range	Groq Calls	On Primary Exhaustion
Key #1	Queries 1-20	40 calls (2x20)	Falls back to keys #6, #7, ... in order
Key #2	Queries 21-40	40 calls (2x20)	Falls back to keys #6, #7, ... in order
Key #3	Queries 41-60	40 calls (2x20)	Falls back to keys #6, #7, ... in order
Key #4	Queries 61-80	40 calls (2x20)	Falls back to keys #6, #7, ... in order
Key #5	Queries 81-100	40 calls (2x20)	Falls back to keys #6, #7, ... in order

9. Key Dependencies

Library	Version	Purpose
chromadb	>=0.5.0	Persistent vector store with HNSW index
sentence-transformers	>=3.0.0	all-MiniLM-L6-v2 embedding model

Library	Version	Purpose
groq	>=0.11.0	Groq API client — used for both expansion and generation
pandas / numpy	>=2.0	Data manipulation
ragas	>=0.2.0	RAGAS evaluation framework
deepeval	>=1.0.0	DeepEval evaluation framework
scikit-learn	latest	TF-IDF vectoriser and cosine similarity for metrics
openpyxl	>=3.1.0	Excel export for evaluation results
reportlab	>=4.0.0	PDF report generation
python-dotenv	>=1.0.0	Environment variable management

10. Configuration Reference

Config Key	Default	Description
GROQ_API_KEYS	env var	Comma-separated list of Groq API keys
EMBEDDING_MODEL	all-MiniLM-L6-v2	SentenceTransformer model for ChromaDB indexing
GROQ_MODEL	llama-3.3-70b-versatile	LLM for both query expansion and generation
COLLECTION_NAME	ecommerce_kb	ChromaDB collection identifier
CHROMA_DB_PATH	chroma_db/	Filesystem path for persistent vector store
NUM_QUERY_VARIANTS	4	Total query variants including the original
PER_QUERY_TOP_K	10	Documents retrieved per query variant
FINAL_TOP_K	5	Documents returned after RRF fusion
RRF_K	60	RRF smoothing constant (standard value)
EXPANDER_TEMPERATURE	0.7	LLM temperature for query expansion

11. Summary

Multi-Query RAG addresses the vocabulary mismatch problem in standard dense retrieval by diversifying the query formulation before retrieval. The four-stage pipeline (expand → multi-retrieve → RRF fuse → generate) ensures that relevant documents are retrieved even when the user's exact phrasing does not match the knowledge base vocabulary. Reciprocal Rank Fusion provides a theoretically grounded, parameter-light mechanism to merge multiple ranked lists, with the $k=60$ constant smoothing the contribution of top-rank documents relative to lower-rank appearances across query variants. The cost is 2 Groq calls per query and higher inter-query delays in evaluation, but the benefit is a broader semantic coverage of the knowledge base per query.